

小办·单次问答成本账+缓存策略

基于小办真实工作流（LLM调用点逐节点拆）·价格按公开报价估算（量级参考，非实际账单）
更新：2026-09-08

1.一句话结论

小办一次正常问答的模型成本约4-5分钱（豆包现网）；换DeepSeek自托管后端约1.5分；加上「标准QA短路+前缀缓存+结果缓存」可以压到1分以内。
成本大头不是改写、不是拆分，而是答案生成节点的输出token +大system prompt的输入token——这两点决定了降本要从哪下手。

2.模型与单价（公开报价，非实际账单）

模型	用途（节点）	输入元/百万	输出元/百万	说明
doubao-seed-2.0-pro	拆分类/答案生成/兜底	≈ 3.2-6.4	≈ 16-32	火山引擎刊例按输入长度分档；低价档为部分渠道/时段价
doubao-seed-2.0-lite	问题改写（规范化任务用便宜的）	≈ 0.6-1.2	≈ 3.6-7.2	改写只做规范化，不需要pro
deepseek-v3.2（百炼）	自托管后端主模型	2	3	输出价比豆包pro低一个量级；缓存命中输入0.4元/M

[估算] 口径：价格为2026-07前后火山引擎/DeepSeek公开报价的量级参考，实际以控制台账单为准。
我的目的不是报精确分，而是展示成本结构从哪来、能往哪省。

3. LLM调用点拆解（先看钱花在哪）

基于xiaoban-workflow-src/config/*.json的真实system prompt长度（已估算token）：

节点	模型	System Prompt	动态输入	输出（估）	是否每次必走
query_rewrite问题改写	lite	~1500	~25	~40	是
split_and_classify拆分类	pro	~1470	~15	~120	是
normal_answer答案生成	pro	~2320	检索片段 ~1000	~400	是（含标准QA命中时）
fallback_answer兜底	pro	~330	~30	~50	仅兜底时（约8%用例）

关键发现1：standard_hit=true（标准QA命中）时normal_answer仍会调一次pro——提示词让它"直接封装预制答案"，但没有在代码层短路。这是第一个该省的调用。
关键发现2：每条链路的system prompt约5300 token是固定的（可走前缀缓存），只有改写输入、检索片段是动态的。

4.两条典型链路的单次成本

链路A：标准QA命中（快路径）

调用	in token	out	豆包cost (pro 6.4 / lite 1.2元/M)
rewrite(lite)	1525	40	0.0018 + 0.0003 ≈ 0.002元
split(pro)	1485	120	0.0095 + 0.0038 ≈ 0.013元
normal(pro)	2400	250	0.0154 + 0.0080 ≈ 0.023元
合计			≈ 0.038元 (约3.8分)

链路B：未命中标准QA（完整检索->回答）

调用	in token	out	豆包cost
rewrite(lite)	1525	40	≈ 0.002元
split(pro)	1485	120	≈ 0.013元
normal(pro)	3420	400	0.0219 + 0.0128 ≈ 0.035元
合计			≈ 0.050元 (约5分)

按真实分布加权（L1命中约20%+，其余走检索）：单次成本≈ 0.045-0.05元。
若走DeepSeek自托管后端（2/3元每百万）：链路B ≈ 0.014元（1.4分），比豆包pro省~70%。

5.规模化估算（量级感）

日问答量	月问答量	豆包现网 (≈0.047/次)	DeepSeek后端 (≈0.014/次)	+缓存优化后
200次/天（小团队）	4.4千次	≈ 210元	≈ 62元	≈ 40元
1,000次/天	2.2万次	≈ 1,030元	≈ 310元	≈ 180元
5,000次/天（千人工活跃用）	11万次	≈ 5,170元	≈ 1,540元	≈ 900元

量级结论：这不是一个"烧钱"的应用，月成本千元级即可服务千人工；
但优化前后差5-10倍，对B端报价（按量计费or订阅）有实际意义。

6.缓存与降本策略（按收益排序）

①标准QA命中->代码层短路（最高优先，改动最小）

做法：standard_hit=true时，normal_answer不再调LLM，由代码直接封装预制答案返回。
省什么：省一次pro调用（链路A从3.8分->1.6分，-58%）。
代价：命中越多越值；L1高频直答本来就是"为了省一次大模型调用"设计的（63条库的意义）。

② system prompt前缀缓存（大模型厂商自带，几乎零改造成本）

做法：利用火山引擎/DeepSeek的前缀缓存（cache input约为正常输入价1/10）。
省什么：链路B中~5300 token固定system prompt输入成本降到1/10 -> 单次约省0.012元。
注意：要求system prompt 开头固定（把常变的检索片段放到user消息尾部，不要插在system中间）。

③整链结果缓存（同问直接返回，边际成本归零）

做法：对「改写后问句hash -> 标准答案/检索答案」做一层Redis缓存；命中即整体跳过。

命中场景：企业内制度问答重复率高（同一问题多人问、同一人反复问）。

省什么：命中时整链0 LLM成本。

④模型分层（已在做，继续坚持）

做法：改写/拆分用lite、回答用pro（目前config已是这样）。

原则：确定性任务用便宜模型+强约束，把pro留给"要组织语言"的生成任务。

延伸：兜底话术是固定文本，可以干脆不调LLM（或降到mini），代码直接返回。

⑤控制输出token（成本最容易被忽略的变量）

做法：答案节点max_completion_tokens与输出格式约束（三要素固定结构、不啰嗦）。

为什么：pro输出价是输入的5-10倍（32 vs 6.4元/百万），输出每省100 token ≈省0.003元。

连带收益：输出短->延迟低->体验好。

7.核心结论

我算过小办单次问答的成本：链路里最贵的是答案生成节点——它每次都带一份几千token的system prompt，输出又是最贵的计费项。

三条降本：第一，标准QA命中时在代码层短路、不再调大模型，省58%；

第二，system prompt固定部分走前缀缓存，输入价降一个量级；

第三，企业问答重复率高，同问缓存命中后边际成本归零。

结论：千人员工月成本千元级，B端按量报价完全站得住。

问：你的"91.5%"和成本是什么关系？

命中率越高越省钱——L1高频直答（100%）本来就是"缓存式回答、省一次大模型调用"。

所以评测体系不只是质量指标，它直接决定成本结构：分层越高分、降本空间越大。

问：为什么不用开源模型本地部署？

DeepSeek-V3.2开源、API已经比豆包便宜一个量级（输出3 vs 32元/百万），

量再大可以考虑本地化：单条GPU卡几百token/s，要自己扛运维和容量，

属于"月调用超百万级再算账"的事——现在这个量级，API更划算。

8.假设与口径（诚实披露）

- Token估算：按system prompt中文字符≈ 1 token/字折算，未计对话历史（当前是单轮问答，无多轮记忆）。
- 输出token为估算区间（改写40 / 拆分120 / 回答250-400），建议接入用量日志后按真实值替换。
- 价格为公开刊例的量级参考（豆包pro输入3.2-6.4、输出16-32元/百万存在渠道与时段差异）。
- 未计：知识库检索调用费（量级很小）、向量库存储、飞书消息通道费。
- 下一步建议：在LLMClient.invoke处加一行token用量打点（usage字段），一周后拿真实分布替换估算——这就是"成本意识落地"的完整闭环。